Storage Classes

This document describes the concept of a StorageClass in Kubernetes. Familiarity with volumes and persistent volumes is suggested.

A StorageClass provides a way for administrators to describe the _classes_ of storage they offer. Different classes might map to quality-of-service levels, or to backup policies, or to arbitrary policies determined by the cluster administrators. Kubernetes itself is unopinionated about what classes represent.

The Kubernetes concept of a storage class is similar to “profiles” in some other storage system designs.

StorageClass objects

Each StorageClass contains the fields provisioner, parameters, and reclaimPolicy, which are used when a PersistentVolume belonging to the class needs to be dynamically provisioned to satisfy a PersistentVolumeClaim (PVC).

The name of a StorageClass object is significant, and is how users can request a particular class. Administrators set the name and other parameters of a class when first creating StorageClass objects.

As an administrator, you can specify a default StorageClass that applies to any PVCs that don't request a specific class. For more details, see the PersistentVolumeClaim concept.

Here's an example of a StorageClass:

Default StorageClass

You can mark a StorageClass as the default for your cluster. For instructions on setting the default StorageClass, see Change the default StorageClass.

When a PVC does not specify a storageClassName, the default StorageClass is used.

If you set the storageclass.kubernetes.io/is-default-class annotation to true on more than one StorageClass in your cluster, and you then create a PersistentVolumeClaim with no storageClassName set, Kubernetes uses the most recently created default StorageClass.

Note: You should try to only have one StorageClass in your cluster that is marked as the default. The reason that Kubernetes allows you to have multiple default StorageClasses is to allow for seamless migration.

You can create a PersistentVolumeClaim without specifying a storageClassName for the new PVC, and you can do so even when no default StorageClass exists in your cluster. In this case, the new PVC creates as you defined it, and the storageClassName of that PVC remains unset until a default becomes available.

You can have a cluster without any default StorageClass. If you don't mark any StorageClass as default (and one hasn't been set for you by, for example, a cloud provider), then Kubernetes cannot apply that defaulting for PersistentVolumeClaims that need it.

If or when a default StorageClass becomes available, the control plane identifies any existing PVCs without storageClassName. For the PVCs that either have an empty value for storageClassName or do not have this key, the control plane then updates those PVCs to set storageClassName to match the new default StorageClass. If you have an existing PVC where the storageClassName is "", and

you configure a default StorageClass, then this PVC will not get updated.

In order to keep binding to PVs with storageClassName set to "" (while a default StorageClass is present), you need to set the storageClassName of the associated PVC to "".

Provisioner

Each StorageClass has a provisioner that determines what volume plugin is used for provisioning PVs. This field must be specified.

Volume Plugin	Internal Provisioner	Config Example
AzureFile	Yes	Azure File
CephFS	-	-
FC	-	-
FlexVolume	-	-
iSCSI	-	-
Local	-	Local
NFS	-	NFS
PortworxVolume	Yes	Portworx Volume
RBD	-	Ceph RBD
VsphereVolume	Yes	vSphere

You are not restricted to specifying the "internal" provisioners listed here (whose names are prefixed with "kubernetes.io" and shipped alongside Kubernetes). You can also run and specify external provisioners, which are independent programs that follow a specification defined by Kubernetes. Authors of external provisioners have full discretion over where their code lives, how the provisioner is shipped, how it needs to be run, what volume plugin it uses (including Flex), etc. The repository kubernetes-sigs/sig-storage-lib-external-provisioner houses a library for writing external provisioners that implements the bulk of the specification. Some external provisioners are listed under the repository kubernetes-sigs/sig-storage-lib-external-provisioner.

For example, NFS doesn't provide an internal provisioner, but an external provisioner can be used. There are also cases when 3rd party storage vendors provide their own external provisioner.

Reclaim policy

PersistentVolumes that are dynamically created by a StorageClass will have the reclaim policy specified in the reclaimPolicy field of the class, which can be either Delete or Retain. If no reclaimPolicy is specified when a StorageClass object is created, it will default to Delete.

PersistentVolumes that are created manually and managed via a StorageClass will have whatever reclaim policy they were assigned at creation.

Volume expansion

PersistentVolumes can be configured to be expandable. This allows you to resize the volume by editing the corresponding PVC object, requesting a new larger amount of storage.

The following types of volumes support volume expansion, when the underlying StorageClass has the field allowVolumeExpansion set to true.

Volume type	Required Kubernetes version for volume expansion
Azure File	1.11
CSI	1.24
FlexVolume	1.13
Portworx	1.11
rbd	1.11

Note: You can only use the volume expansion feature to grow a Volume, not to shrink it.

Mount options

PersistentVolumes that are dynamically created by a StorageClass will have the mount options specified in the mountOptions field of the class.

If the volume plugin does not support mount options but mount options are specified, provisioning will fail. Mount options are not validated on either the class or PV. If a mount option is invalid, the PV mount fails.

Volume binding mode

The volumeBindingMode field controls when volume binding and dynamic provisioning should occur. When unset, Immediate mode is used by default.

The Immediate mode indicates that volume binding and dynamic provisioning occurs once the PersistentVolumeClaim is created. For storage backends that are topology-constrained and not globally accessible from all Nodes in the cluster, PersistentVolumes will be bound or provisioned without knowledge of the Pod's scheduling requirements. This may result in unschedulable Pods.

A cluster administrator can address this issue by specifying the WaitForFirstConsumer mode which will delay the binding and provisioning of a PersistentVolume until a Pod using the PersistentVolumeClaim is created. PersistentVolumes will be selected or provisioned conforming to the topology that is specified by the Pod's scheduling constraints. These include, but are not limited to, resource requirements, node selectors, pod affinity and anti-affinity, and taints and tolerations.

The following plugins support WaitForFirstConsumer with dynamic provisioning:

- CSI volumes, provided that the specific CSI driver supports this

The following plugins support WaitForFirstConsumer with pre-created PersistentVolume binding:

- CSI volumes, provided that the specific CSI driver supports this
- local

Note: If you choose to use WaitForFirstConsumer, do not use nodeName in the Pod spec to specify node affinity. If nodeName is used in this case, the scheduler will be bypassed and PVC will remain in pending state. Instead, you can use node selector for kubernetes.io/hostname:

Allowed topologies

When a cluster operator specifies the WaitForFirstConsumer volume binding mode, it is no longer necessary to restrict provisioning to specific topologies in most situations. However, if still required, allowedTopologies can be specified.

This example demonstrates how to restrict the topology of provisioned volumes to specific zones and should be used as a replacement for the zone and zones parameters for the supported plugins.

Parameters

StorageClasses have parameters that describe volumes belonging to the storage class. Different parameters may be accepted depending on the provisioner. When a parameter is omitted, some default is used.

There can be at most 512 parameters defined for a StorageClass. The total length of the parameters object including its keys and values cannot exceed 256 KiB.

AWS EBS

Kubernetes 1.35 does not include a awsElasticBlockStore volume type.

The AWSElasticBlockStore in-tree storage driver was deprecated in the Kubernetes v1.19 release and then removed entirely in the v1.27 release.

The Kubernetes project suggests that you use the AWS EBS out-of-tree storage driver instead.

Here is an example StorageClass for the AWS EBS CSI driver:

tagSpecification: Tags with this prefix are applied to dynamically provisioned EBS volumes.

AWS EFS

To configure AWS EFS storage, you can use the out-of-tree AWS_EFS_CSI_DRIVER.

- provisioningMode: The type of volume to be provisioned by Amazon EFS. Currently, only access point based provisioning is supported (efs-ap).
- fileSystemId: The file system under which the access point is created.
- directoryPerms: The directory permissions of the root directory created by the access point.

For more details, refer to the AWS_EFS_CSI_Driver Dynamic Provisioning documentation.

NFS

To configure NFS storage, you can use the in-tree driver or the NFS CSI driver for Kubernetes (recommended).

- server: Server is the hostname or IP address of the NFS server.
- path: Path that is exported by the NFS server.
- readOnly: A flag indicating whether the storage will be mounted as read only (default false).

Kubernetes doesn't include an internal NFS provisioner. You need to use an external provisioner to create a StorageClass for NFS. Here are some examples:

- NFS Ganesha server and external provisioner
- NFS subdir external provisioner

vSphere

There are two types of provisioners for vSphere storage classes:

- CSI provisioner: csi.vsphere.vmware.com
- vCP provisioner: kubernetes.io/vsphere-volume

In-tree provisioners are deprecated. For more information on the CSI provisioner, see Kubernetes vSphere CSI Driver and vSphereVolume CSI migration.

CSI Provisioner

The vSphere CSI StorageClass provisioner works with Tanzu Kubernetes clusters. For an example, refer to the vSphere CSI repository.

vCP Provisioner

The following examples use the VMware Cloud Provider (vCP) StorageClass provisioner.

- Create a StorageClass with a user specified disk format.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: fast
provisioner: kubernetes.io/vsphere-volume
parameters:
diskformat: zeroedthick
```

diskformat: thin, zeroedthick and eagerzeroedthick. Default: "thin".

- Create a StorageClass with a disk format on a user specified datastore.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: fast
provisioner: kubernetes.io/vsphere-volume
parameters:
diskformat: zeroedthick
datastore: VSANDatastore
```

datastore: The user can also specify the datastore in the StorageClass. The volume will be created on the datastore specified in the StorageClass, which in this case is VSANDatastore. This field is optional. If the datastore is not specified, then the volume will be created on the datastore specified in the vSphere config file used to initialize the vSphere Cloud Provider.

- Storage Policy Management inside kubernetes
- Using existing vCenter SPBM policy

One of the most important features of vSphere for Storage Management is policy based Management. Storage Policy Based Management (SPBM) is a storage policy framework that provides a single unified control plane across a broad range of data services and storage solutions. SPBM enables vSphere administrators to overcome upfront storage provisioning challenges, such as capacity planning, differentiated service levels and managing capacity headroom.

The SPBM policies can be specified in the StorageClass using the storagePolicyName parameter.

- Virtual SAN policy support inside Kubernetes

Vsphere Infrastructure (VI) Admins will have the ability to specify custom Virtual SAN Storage Capabilities during dynamic volume provisioning. You can now define storage requirements, such as performance and availability, in the form of storage capabilities during dynamic volume provisioning. The storage capability requirements are converted into a Virtual SAN policy which are then pushed down to the Virtual SAN layer when a persistent volume (virtual disk) is being created. The virtual disk is distributed across the Virtual SAN datastore to meet the requirements.

You can see Storage Policy Based Management for dynamic provisioning of volumes for more details on how to use storage policies for persistent volumes management.

Ceph RBD (deprecated)

Note: This internal provisioner of Ceph RBD is deprecated. Please use CephFS RBD CSI driver.

- monitors: Ceph monitors, comma delimited. This parameter is required.
- adminId: Ceph client ID that is capable of creating images in the pool. Default is "admin".
- adminSecretName: Secret Name for adminId. This parameter is required. The provided secret must have type "kubernetes.io/rbd".
- adminSecretNamespace: The namespace for adminSecretName. Default is "default".
- pool: Ceph RBD pool. Default is "rbd".
- userId: Ceph client ID that is used to map the RBD image. Default is the same as adminId.
- userSecretName: The name of Ceph Secret for userId to map RBD image. It must exist in the same namespace as PVCs. This parameter is required. The provided secret must have type "kubernetes.io/rbd", for example created in this way:

```
kubectl create secret generic ceph-secret --type="kubernetes.io/rbd" \
--from-literal=key='QVFEQ1pMdFhPUnQrSmhBQUFYaERWNHJsZ3BsMmNjcDR6RFZST0E9PQ==' \
--namespace=kube-system
```

- userSecretNamespace: The namespace for userSecretName.
- fsType: fsType that is supported by kubernetes. Default: "ext4".
- imageFormat: Ceph RBD image format, "1" or "2". Default is "2".
- imageFeatures: This parameter is optional and should only be used if you set imageFormat to "2". Currently supported features are layering only. Default is "", and no features are turned on.

Azure Disk

Kubernetes 1.35 does not include a azureDisk volume type.

The azureDisk in-tree storage driver was deprecated in the Kubernetes v1.19 release and then removed entirely in the v1.27 release.

The Kubernetes project suggests that you use the Azure Disk third party storage driver instead.

Azure File (deprecated)

- skuName: Azure storage account SKU tier. Default is empty.
- location: Azure storage account location. Default is empty.
- storageAccount: Azure storage account name. Default is empty. If a storage account is not provided, all storage accounts associated with the resource group are searched to find one that matches skuName and location. If a storage account is provided, it must reside in the same resource group as the cluster, and skuName and location are ignored.
- secretNamespace: the namespace of the secret that contains the Azure Storage Account Name and Key. Default is the same as the Pod.
- secretName: the name of the secret that contains the Azure Storage Account Name and Key. Default is azure-storage-account-<accountName>-secret

- readOnly: a flag indicating whether the storage will be mounted as read only. Defaults to false which means a read/write mount. This setting will impact the ReadOnly setting in VolumeMounts as well.

During storage provisioning, a secret named by secretName is created for the mounting credentials. If the cluster has enabled both RBAC and Controller Roles, add the create permission of resource secret for clusterrole system:controller:persistent-volume-binder.

In a multi-tenancy context, it is strongly recommended to set the value for secretNamespace explicitly, otherwise the storage account credentials may be read by other users.

Portworx volume (deprecated)

- fs: filesystem to be laid out: none/xfs/ext4 (default: ext4).
- block_size: block size in Kbytes (default: 32).
- repl: number of synchronous replicas to be provided in the form of replication factor 1..3 (default: 1) A string is expected here i.e. "1" and not 1.
- priority_io: determines whether the volume will be created from higher performance or a lower priority storage high/medium/low (default: low).
- snap_interval: clock/time interval in minutes for when to trigger snapshots. Snapshots are incremental based on difference with the prior snapshot, 0 disables snaps (default: 0). A string is expected here i.e. "70" and not 70.
- aggregation_level: specifies the number of chunks the volume would be distributed into, 0 indicates a non-aggregated volume (default: 0). A string is expected here i.e. "0" and not 0
- ephemeral: specifies whether the volume should be cleaned-up after unmount or should be persistent. emptyDir use case can set this value to true and persistent volumes use case such as for databases like Cassandra should set to false, true/false (default false). A string is expected here i.e. "true" and not true.

Local

Local volumes do not support dynamic provisioning in Kubernetes 1.35; however a StorageClass should still be created to delay volume binding until a Pod is actually scheduled to the appropriate node. This is specified by the WaitForFirstConsumer volume binding mode.

Delaying volume binding allows the scheduler to consider all of a Pod's scheduling constraints when choosing an appropriate PersistentVolume for a PersistentVolumeClaim.